AI Interview & Preparation Platform

Complete Technical Architecture, Implementation Analysis & Code Guide

Author / Assistant: Antigravity

Date: September 2026

Architecture: FastAPI + SQLite + HTML5/CSS/JS

1. Executive Overview & System Purpose

The **AI Interview & Preparation Platform** is a multi-user, production-ready web application that automates technical interview simulation, skill matching, study document distribution, cheatsheet generation, and candidate evaluation. The project is cleanly structured into a modular **Backend** (FastAPI, SQLite, LangChain, Groq LLM, Faster-Whisper, Kokoro TTS, ReportLab) and a **Frontend** (modern HTML5, CSS3, vanilla JavaScript) strictly designed with non-neon, professional aesthetics.

Users can register separate accounts, upload resumes (.pdf/.docx), optionally analyze a Job Description for skill matching, launch a dedicated interview room with automated voice recording and live speech-to-text, receive instant AI feedback on their actual answers, archive permanent interview histories in SQL, explore study documents in a PDF reader, and compile personalized cheat sheets into downloadable PDFs.

2. Project Directory Organization

Directory / File	Role & Component Purpose	Status
backend/app.py	FastAPI application serving REST APIs and static frontend routes.	NEW
database.py	SQLAlchemy engine, declarative base, and session generator.	MODIFIED
data_space.py	SQLAlchemy models: User, UserProfile, and UserHistory.	MODIFIED
report.py	Interview assessment model and LLM summary evaluation.	MODIFIED
genrating_cheatsheet.py	Parses LLM cheatsheet output and compiles formatted PDF.	MODIFIED
main.py	Root executable that starts the Uvicorn server.	MODIFIED
run.bat	Windows one-click batch launcher using the project's .venv.	NEW
frontend/css/style.css	Non-neon styling system, card layout, tags, and progress bars.	NEW
frontend/login.html	Multi-user authentication portal (Sign In vs Register tabs).	NEW
frontend/index.html	Home page with document loader, JD matcher, and interview launcher.	NEW
frontend/interview.html	Dedicated interview room with live speech STT and coaching feedback.	NEW
frontend/prep.html	Preparation Hub listing study documents and cheatsheet trigger.	NEW
frontend/profile.html	Candidate profile view (excluding password) and SQL history table.	NEW
frontend/js/*.js	Client-side logic: api.js, app.js, interview.js, prep.js, profile.js.	NEW
doc_loader.py	LangChain PDF/DOCX document loader and regex text normalizer.	EXISTING
skill_matcher.py	Skill taxonomy matcher between resume and job description.	EXISTING
llm.py	Groq LLM chains for question generation, evaluation, and cheatsheet.	EXISTING
documents/	Storage folder for study PDFs, generated cheatsheets, and docs.	EXISTING

3. Detailed File-by-File Analysis

Below is a comprehensive breakdown of every file modified or created, explaining what it was before, what it is now, why the changes were made, what the code means, and how it works.

3.1. backend/app.py [NEW FILE]

Before: Did not exist. The project only had disconnected standalone CLI scripts (`interview_flow.py`, `test.py`, `prep.py`, `check_cheatsheet.py`) that operated interactively in the terminal with hardcoded file paths and no web server.

Now: A central FastAPI application that ties the entire system into a cohesive multi-user web platform. It exposes RESTful APIs and serves static HTML/CSS/JS frontend pages.

Why Created: To provide an asynchronous web server that coordinates multi-user registration, secure JWT/HMAC token sessions, document parsing, skill matching, live interview evaluation, cheatsheet generation, and SQL database transactions.

Key Code Components & What They Mean:

• create_token(username) & verify_token(token): Creates HMAC-SHA256 signed session tokens so users stay authenticated without external heavy dependencies.
• get_current_user(...): FastAPI dependency that parses the Authorization: Bearer <token> header, verifies signature, and fetches the user from SQLite.
• POST /api/auth/register: Checks if username exists, hashes password with salt, creates user record in users table, initializes empty UserProfile, and returns session token.
• POST /api/auth/login: Validates credentials. Rejects unregistered users with a 400 error requiring registration first (strict separation).
• POST /api/resume/upload: Accepts PDF/DOCX file uploads, saves them per user, calls load_document() & cleaning(), extracts skills, and updates UserProfile.resume_skill in SQL.
• POST /api/skills/match: Runs skill_matcher.extraction() to compute role match score %, matching skills, and missing skills, storing them in SQL.
• POST /api/interview/start: Generates technical interview questions via llm.genrate_ques() with automated fallbacks so interview never stalls.
• POST /api/interview/evaluate: Evaluates candidate's actual answer using llm.evaluate_ans(), providing immediate feedback on correctness and depth.
• POST /api/interview/suggest_answer: On-demand AI answer generator when microphone is not detected or user requests an answer draft.
• POST /api/interview/finish: Calls report.generate_summary() to compute final assessment (Score, Accuracy %, Blunders, Confidence, Streak, Communication) and permanently commits a new row into the SQLite user_history table.
• GET /api/documents/* & POST /api/cheatsheet/generate: Lists preparation documents, serves inline PDF streams, and generates custom cheatsheets via genrating_cheatsheet.build_cheatsheet_pdf().
• GET /api/profile & GET /api/profile/history: Queries SQLite and returns candidate profile details (strictly omitting password) and all past interview history records.

3.2. database.py [MODIFIED]

Before: Used relative path sqlite:///./interview.db and default engine arguments without threading protection. Running from subdirectories would create duplicate database files in different folders.

Now: Resolves BASE_DIR = os.path.dirname(os.path.abspath(__file__)) and sets SQLALCHEMY_DATABASE_URL = f'sqlite:///{DB_PATH}' with connect_args={'check_same_thread': False}.

Why Changed: To guarantee that SQLite always references the single canonical interview.db in the project root regardless of the execution working directory, and prevents multi-threaded FastAPI requests from crashing on SQLite's default single-thread lock.

Meaning of Code: SessionLocal creates isolated transactional database sessions, and get_db() is a generator that yields a session and guarantees it is cleanly closed upon request completion.

3.3. data_space.py [MODIFIED]

Before: Contained basic SQLAlchemy models (User, UserProfile, UserHistory) without table indexing, timestamps, or an automated table creation routine.

Now: Added column indexes (index=True) on username and foreign keys, added created_at = Column(String, default=lambda: datetime.now().strftime('%Y-%m-%d %H:%M')) to UserHistory, and introduced the init_db() function that invokes Base.metadata.create_all(bind=engine).

Why Changed: To ensure that interview histories record exact timestamps, queries run efficiently across multiple users, and database tables are automatically initialized without manual SQL commands.

Meaning of Code: User stores credentials; UserProfile stores skills, target role, and match score per user; UserHistory records each completed interview session with score, accuracy, blunders, confidence, streak, and communication notes.

3.4. report.py [MODIFIED]

Before: InterviewReport strictly typed Score: int and Accuracy: int. If Groq LLM returned floats (e.g. 85.0) or non-standard JSON formatting, Pydantic parser would throw an unhandled exception and abort the interview.

Now: Types changed to Union[int, float] with defaults. In generate_summary(), the LLM invocation is wrapped in a robust try...except block that logs any LLM error and provides a structured fallback report based on candidate answer volume.

Why Changed: To ensure that interview evaluation and SQL archiving NEVER fail, guaranteeing that the user's interview history is always reliably stored in SQLite even during network hiccups.

3.5. genrating_cheatsheet.py [MODIFIED]

Before: Contained hardcoded top-level script code at lines 8-10 (load_document(r'C:\Users\hp\Downloads\Resume_in_pdf.pdf')) and at line 377 (build_cheatsheet_pdf(data, 'documents/My_cheatsheet.pdf')). If imported, it would attempt to read a hardcoded file path and crash if missing, or run unwanted PDF compilation during server startup.

Now: Wrapped all script execution inside standard if __name__ == '__main__': guards.

Why Changed: Enables backend/app.py to safely import build_cheatsheet_pdf as a reusable library function for web API requests without accidental script execution on startup.

3.6. main.py & run.bat [MODIFIED & NEW]

Before: main.py only contained print('Hello from ai-interview!').

Now: main.py automatically resolves the project path and launches Uvicorn: uvicorn.run('backend.app:app', host='127.0.0.1', port=8000, reload=True). Additionally, run.bat was created to allow one-click launch using the project's virtual environment (.\.venv\Scripts\python.exe).

Why Changed: Ensures the server starts cleanly on Windows without conflicts with Microsoft Store Python or missing PATH environment variables.

3.7. frontend/css/style.css [NEW FILE]

Before: No CSS styling existed.

Now: A comprehensive, cohesive modern design system built with CSS variables. Uses neutral slates (#f8fafc, #f1f5f9), deep slate text (#0f172a), calm royal blue (#2563eb), soft card borders (#e2e8f0), and muted status pills. Strictly avoids neon colors as requested.

Why Created: Provides a professional, distraction-free corporate visual identity for all pages, modals, tables, badges, and the live interview room.

3.8. frontend/js/api.js [NEW FILE]

Before: Did not exist.

Now: A unified client-side API helper that manages authentication state (interview_token in localStorage), injects Authorization: Bearer <token> into all outgoing requests, handles JSON serialization, manages file uploads via FormData, and automatically redirects to /login if a 401 Unauthorized status occurs.

3.9. frontend/login.html [NEW FILE]

Before: Did not exist.

Now: Clean dual-tab authentication card for Sign In and Register. Strictly separates sign-in from registration so users cannot sign in with non-existent accounts without registering first.

3.10. frontend/index.html & frontend/js/app.js [NEW FILES]

Before: Did not exist.

Now: The Home Page of the platform featuring:

• **Navbar:** Navigation links strictly limited to *Home*, *Candidate Profile*, and *Preparation* (no Interview Room link in navbar, as requested).
• **Resume Loader:** Drag-and-drop dropzone supporting PDF/DOCX. Automatically uploads and parses skills into interactive blue tags.
• **Skill Matcher (Optional):** Allows candidate to paste a Job Description and target role, displaying a calculated Match Score %, matching skills (green), and missing skills (red).
• **Launch Interview Room Button:** Sets resume/JD context into localStorage and navigates immediately (0ms delay) to /interview.

3.11. frontend/interview.html & frontend/js/interview.js [NEW FILES]

Before: Did not exist. All interview logic was trapped in terminal CLI scripts.

Now: The dedicated, immersive interview environment featuring:

• **Instant Question Display:** Question 1 is visible immediately on page load with zero blank screen or waiting delay. Custom questions load seamlessly in the background.
• **Question Audio (TTS):** Uses browser SpeechSynthesis to read each question aloud to the candidate.
• **Automated Voice Recording:** The exact instant the question finishes being spoken (via utterance.onend with a browser safety fallback), the microphone **automatically turns on and begins recording** without requiring manual clicks.
• **Live Subtitle Strip:** Real-time speech-to-text displays every word as the candidate speaks with a pulsing recording indicator, streaming text live into the answer box.
• **Auto-write Suggested Answer Option:** A one-click helper (■ Auto-write Suggested Answer) that uses AI to draft a strong candidate response if voice is not detected or microphone is disabled.
• **Real Evaluation:** Submits the candidate's exact answer to llm.evaluate_ans(), providing genuine, personalized coaching feedback.
• **Guaranteed SQL History Archiving:** When the candidate finishes all questions (or clicks *Complete & Save Report*

at any time), the assessment summary is generated and committed to the SQLite user_history table.

3.12. frontend/prep.html & frontend/js/prep.js [NEW FILES]

Before: Terminal script prep.py printed document names in console and opened them via os.startfile().

Now: Web-based Preparation Hub displaying all study documents in the documents/ directory with file sizes and modification timestamps. Includes an inline PDF preview modal (iframe) so users can study without leaving the browser, direct download buttons, and a small **Generate Cheatsheet** button that compiles a personalized PDF cheat sheet from the user's resume using genrating_cheatsheet.py.

3.13. frontend/profile.html & frontend/js/profile.js [NEW FILES]

Before: profile_data.py saved static JSON to profile.json.

Now: Dynamic candidate profile page that loads all candidate metrics from SQLite (username, editable target role, role match score %, resume skills, matching skills, missing skills) — **strictly excluding password** as requested. At the bottom, it queries and displays the complete **Interview History** from the SQL user_history table (Score/100, Accuracy %, Blunders, Confidence level, Streak, and Communication assessment with exact dates and times).

4. SQLite Database Architecture (interview.db)

The database contains three interconnected tables managed by SQLAlchemy in data_space.py:

Table Name	Columns & Data Types	Description & Constraints
users	id (INTEGER PK) username (STRING UNIQUE) password (STRING HASHED)	User authentication table. Passwords are salted SHA-256 hashes.
user_profile	id (INTEGER PK) username (FK -> users.username) target_role (STRING) resume_skill (STRING JSON) matching_skill (STRING JSON) missing_skill (STRING JSON) score (INTEGER)	Candidate profile inventory. Stores extracted skills and role match score. Password is never included.
user_history	id (INTEGER PK) username (FK -> users.username) score (INTEGER) accuracy (FLOAT) blunder (INTEGER) confidence (STRING) streak (INTEGER) communication (STRING) created_at (STRING TIMESTAMP)	Permanent interview assessment history archive. Generated by report.py upon interview completion.

5. End-to-End Execution Flow

• **1. Authentication:** User accesses /login. New candidates register; existing candidates sign in. Server validates credentials and issues an HMAC session token stored in localStorage.

• **2. Setup on Home (/):** Candidate uploads their resume. POST /api/resume/upload parses text and populates resume skills in SQL. Optionally, pasting a Job Description triggers POST /api/skills/match to compute skill alignment.

• **3. Launching Interview:** Clicking *Launch Interview Room* → saves session state to localStorage and immediately navigates to /interview.

• **4. Live Interview Session:** Question 1 appears immediately. AI interviewer speaks the question via TTS. As soon as speech completes, microphone activates automatically with live subtitle transcription. Candidate speaks or types, receives personalized evaluation feedback, and proceeds through questions.

• **5. SQL History Archiving:** Candidate completes interview (or clicks *Complete & Save Report*). Backend generates structured report via report.generate_summary() and inserts a new timestamped row into user_history in SQLite.

• **6. Candidate Profile (/profile):** Displays candidate details (excluding password) and lists all historical interview sessions from SQL.

• **7. Preparation Hub (/prep):** Candidate can review study documents directly in the inline PDF viewer or click *Generate Cheatsheet* to compile an interview cheat sheet PDF.

6. How to Run & Maintain the Application

Option 1: Windows One-Click Launcher
Double-click run.bat in the project root directory.

Option 2: Terminal / PowerShell
Run: .\.venv\Scripts\python.exe -m uvicorn backend.app:app --host 127.0.0.1 --port 8000 --reload

Accessing the Website:
Open your browser and navigate to: http://127.0.0.1:8000

End of Documentation — All systems verified, operational, and persisted in SQL.